Nonstationarity-Aware Mamba for Time Series Forecasting

Draft Manuscript

May 20, 2026

Abstract

Nonstationary time series forecasting is difficult because the data-generating process can change across time. Changes in level, volatility, trend, and temporal regime can make a model trained on past windows unreliable for future horizons. This manuscript studies nonstationarity-aware forecasting with Mamba-based sequence models, using daily GLD close-price prediction as the initial testbed. The proposed model families extend an S-Mamba backbone with adaptive normalization, series decomposition, distribution-shift-aware fusion, and multi-scale Mamba branches. The current manuscript reports only hyperparameter-searched result tracks across 12 horizons from 1 to 360 days. Within the refreshed Optuna-tuned non-Mamba baseline sweep, RMSSE-based evaluation identifies TimesNet as the strongest overall model, iTransformer as a near-tied runner-up, NLinear as the short-horizon leader, PatchTST as the middle-horizon leader, and TimesNet/iTransformer as split long-horizon winners. After Mamba-specific Optuna search, PMS-Mamba-PD becomes the strongest completed Mamba-family model. On the promoted four-horizon summary over 120, 180, 270, and 360 days, PMS-Mamba-PD reaches RMSSE 27.25, RMSE 34.17, and MAE 23.72. This is a 3.40% RMSSE reduction relative to the strongest Optuna-tuned non-Mamba baseline on the same promoted horizon set. Because the current evidence is limited to GLD and the manuscript is still a single-dataset draft, it treats PMS-Mamba-PD as the current strongest completed Mamba result rather than as a universal state-of-the-art claim.

1 Introduction

Long-horizon time series forecasting is important in applications where decisions must be made well before the target is observed, including energy systems, transportation, operations planning, and financial markets [1, 2, 3, 4]. In this setting, forecasting performance depends not only on short-term pattern extraction but also on the ability to preserve useful temporal information across long contexts while remaining stable under changing data distributions. These requirements are difficult to satisfy simultaneously, which is why long-horizon forecasting remains an active research problem across both statistical and deep-learning paradigms [5, 6, 7, 8, 9].

The difficulty is amplified when the underlying series is nonstationary. Real-world financial series can undergo level shifts, volatility changes, trend reversals, and regime-dependent behavior driven by macroeconomic events, policy changes, and evolving market expectations [10]. Under such conditions, a model trained on historical windows may face a materially different prediction problem at test time. This issue is especially relevant for gold-related exchange-traded products such as GLD, whose price dynamics can reflect inflation expectations, risk-off behavior, U.S. dollar movements, and global uncertainty. For this reason, long-horizon GLD forecasting provides a useful testbed for studying whether modern sequence models can remain effective when temporal dependence and distribution shift appear together.

Recent forecasting research has introduced a broad set of architectures for long-context modeling, including efficient Transformers, decomposition-based models, patch-based encoders, and inverted variable-token representations [1, 2, 11, 12, 8]. At the same time, recent studies have shown that explicitly addressing distribution shift and nonstationarity can improve robustness in forecasting pipelines [13, 14]. Mamba adds another attractive direction because it replaces quadratic attention with selective state-space sequence modeling while retaining strong expressive capacity and linear-time scaling [15]. This makes Mamba a promising backbone for long-horizon forecasting, and several recent papers have adapted it to time-series settings [16, 17, 18, 19, 20]. However, efficient sequence modeling alone does not resolve the core difficulty of nonstationary forecasting: level, scale, trend, and residual behavior may still be entangled in a single hidden representation even when the backbone itself is computationally efficient.

This manuscript studies that gap through a nonstationarity-aware Mamba forecasting framework for daily GLD close-price prediction across horizons from 1 to 360 days. The proposed NS-Mamba family preserves an S-Mamba-style backbone while adding adaptive normalization, decomposition, and shift-aware fusion modules designed to expose nonstationary structure before and around sequence modeling. To make comparisons interpretable while still allowing model-specific tuning, the manuscript evaluates an Optuna-tuned non-Mamba baseline sweep alongside a separate Mamba-family Optuna search. This produces a current view of how Transformer-style, decomposition-based, inverted, and Mamba-based models behave after lookback and architecture tuning rather than under one locked default setting.

The current evidence leads to a deliberately narrow conclusion. In the refreshed Optuna-tuned non-Mamba sweep, RMSSE-based evaluation identifies TimesNet as the strongest overall baseline and iTransformer as a near-tied runner-up. NLinear leads the short horizons, PatchTST leads the middle horizons, and TimesNet/iTransformer split the long-horizon wins. Within the Mamba family, the current RMSSE/RMSE/MAE Optuna search identifies PMS-Mamba-PD as the strongest completed Mamba variant on both the shared 120/180/360 subset and the promoted 120/180/270/360 long-horizon summary. Because the current evidence is limited to GLD and this is still a draft, the paper avoids a universal state-of-the-art claim.

The main contributions of this manuscript are as follows:

- it defines a GLD forecasting benchmark spanning 12 prediction horizons from 1 to 360 days, with Optuna-tuned non-Mamba and Mamba-family search tracks;
- it introduces an NS-Mamba architecture family that combines an S-Mamba backbone with adaptive normalization, decomposition, and shift-aware fusion for nonstationary forecasting;
- it reports Optuna-tuned non-Mamba baseline results and Mamba-family search results under RMSSE-based evaluation, with RMSE and MAE retained for price-scale interpretation;
- it identifies PMS-Mamba-PD as the strongest completed Mamba-family configuration in the current hyperparameter search and compares it against the strongest Optuna-tuned non-Mamba RMSSE references.

2 Related Work

2.1 Forecasting Benchmarks and Long-Horizon Evaluation

Forecasting science has long emphasized that model comparisons must be interpreted through the evaluation setting, the forecast horizon, and the diversity of temporal patterns present in the data.

Large empirical studies such as the M4 competition show that forecasting performance is highly dependent on the data-generating context and that no single modeling family dominates across all series and horizons [21]. This observation is relevant to modern deep forecasting because long-horizon tasks combine several difficulties: multi-step error accumulation, changing signal-to-noise ratios across horizons, and sensitivity to distribution shift. Recent surveys and forecasting studies therefore treat long-horizon prediction as a setting in which architecture design, normalization, horizon length, and evaluation protocol must be considered together [4, 3].

The present manuscript follows this forecasting-science view by evaluating a fixed GLD task across horizons from 1 to 360 days rather than reporting a single horizon or a single aggregate score. This choice is important because short-horizon and long-horizon performance can favor different models. It also motivates the use of hyperparameter-searched comparisons rather than a single locked default configuration, because model families can be sensitive to lookback length, normalization, and architecture-specific settings.

2.2 Long-Horizon Neural Forecasting Models

Deep forecasting models have developed along several complementary directions. N-BEATS introduced a neural basis-expansion architecture that remains an important reference point for interpretable and accurate univariate forecasting [5]. N-HiTS extended this direction with hierarchical interpolation and multi-rate signal processing to improve long-horizon performance [6]. These models are not attention-based, but they are important because they show that explicit multi-scale structure can be competitive in long-horizon forecasting.

Transformer-based models introduced another major direction by using attention to model long temporal context. Informer reduces the computational burden of attention for long sequence forecasting [1], while Reformer provides a general efficient-attention design through locality-sensitive hashing and reversible layers [22]. Autoformer and FEDformer add decomposition mechanisms to the forecasting architecture, reflecting the practical value of separating slowly varying components from residual temporal variation [2, 11]. Temporal Fusion Transformers address multi-horizon forecasting with interpretable attention and gating mechanisms, providing a journal-level reference for multi-horizon neural forecasting [3].

More recent work has also questioned whether increasingly complex attention mechanisms are always necessary. DLinear shows that simple decomposition-based linear models can be strong long-term forecasting baselines, especially when evaluation protocols are carefully controlled [7]. PatchTST improves Transformer forecasting by segmenting time series into patches, reducing token length and encouraging local temporal representation learning [8]. TimesNet reformulates temporal variation through two-dimensional representations and provides another strong general-purpose neural forecasting baseline [9]. Together, these studies motivate the baseline set used in this manuscript: the comparison should include not only Mamba models, but also decomposition, patch-based, inverted, and multi-scale alternatives.

2.3 Nonstationarity, Normalization, and Decomposition

Nonstationarity is a central difficulty in long-horizon forecasting because the distribution observed during training may differ from the distribution encountered during testing. Changes in level, scale, trend, volatility, or temporal regime can make a representation learned on past windows unreliable for future windows. This issue is particularly important in financial forecasting, where empirical studies and reviews emphasize the impact of regime changes, noise, and unstable temporal dependence [10].

Several forecasting methods address this problem by modifying either the input distribution or the model architecture. RevIN normalizes each instance and then reverses the normalization at the output, directly targeting distribution shift between input windows and forecast targets [13]. Non-stationary Transformers combine series stationarization with a de-stationary attention mechanism, arguing that forecasting models need both normalized inputs and a way to preserve useful nonstationary information [14]. Decomposition-based methods such as Autoformer, FEDformer, DLinear, and N-HiTS also reflect a related principle: separating trend, seasonal, or multi-scale components can make long-horizon prediction more stable than forcing all variation into a single latent representation [2, 11, 7, 6].

The NS-Mamba family studied here follows this line of work but applies it to a selective state-space backbone. Rather than treating nonstationarity only as a preprocessing issue, the architecture combines reversible scaling, decomposition, and shift-aware fusion around the Mamba encoder. This design is intended to expose level, scale, trend, and residual structure explicitly before those components are mixed by the sequence model.

2.4 Inverted and Variable-Token Forecasting Architectures

Inverted forecasting architectures reconsider the tokenization used by sequence models. iTransformer treats variables as tokens and encodes each variable's temporal history as its representation, showing that this inverted view can be effective for multivariate forecasting [12]. This design is relevant because many forecasting architectures inherit tokenization assumptions from natural language processing, where positions rather than variables are the primary tokens. For time series, however, the choice of tokenization controls how temporal history, cross-variable dependence, and forecast heads interact.

The S_Mamba backbone used in this study follows a related idea by embedding a variable's lookback history before applying Mamba encoder layers. Although the GLD experiment is a single-target forecasting task, the inverted representation remains useful because it defines how the lookback window is compressed into hidden features before horizon projection. This also makes S_Mamba a natural starting point for studying whether explicit nonstationary modules can improve Mamba-based forecasting.

2.5 Mamba-Based Time Series Forecasting

Mamba provides linear-time sequence modeling through selective state spaces, making it attractive for long-context tasks where quadratic attention can be costly [15]. Recent studies have adapted this backbone to time series forecasting in different ways. S_Mamba evaluates whether Mamba is effective for forecasting and uses bidirectional Mamba layers to model inter-variate correlations with feed-forward layers for temporal dependencies [16]. Bi-Mamba+ extends bidirectional processing with a forget gate and relation-aware choices between channel-independent and channel-mixing tokenization [17]. MambaTS analyzes limitations of directly applying Mamba to long-term forecasting and proposes temporal Mamba blocks, variable scanning strategies, and variable-aware scan ordering [18]. TimeMachine uses multiple Mamba pathways to handle channel-mixing and channel-independence settings while preserving linear scalability [19]. TSMamba moves toward a Mamba-based foundation model for time series prediction using bidirectional encoders and transfer learning [20].

These studies show that Mamba has become an active forecasting backbone, but most of them emphasize sequence modeling, scan design, channel mixing, or transfer rather than explicit handling of nonstationarity. The present manuscript is positioned at that intersection. It asks whether

a Mamba forecaster benefits when nonstationary structure is made explicit through adaptive normalization, decomposition, and shift-aware fusion, and it evaluates the completed Mamba Optuna rows with the same RMSSE metric used for the Optuna-tuned non-Mamba baseline leaderboard.

3 Problem Formulation and Method

3.1 Forecasting Task

Let

$$\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L \times C} \tag{1}$$

denote an input window of length L with C variables. The goal is to predict

$$\mathbf{Y}_{t+1:t+H} \in \mathbb{R}^{H \times C_o}, \tag{2}$$

where H is the forecast horizon and C_o is the output dimension. For the GLD study, $C = C_o = 1$ and the target is the daily `close` column. The evaluated horizon set is

$$H \in \{1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360\}. \tag{3}$$

These horizons cover short-term, middle-term, and long-term forecasting behavior.

This manuscript treats nonstationarity as a time-varying relationship between input and output windows. Let $\mathcal{D}_t$ denote the local joint distribution around time t. The nonstationary setting can be written as

$$\mathcal{D}_t(\mathbf{X}_{t-L+1:t}, \mathbf{Y}_{t+1:t+H}) \neq \mathcal{D}_{t+\tau}(\mathbf{X}_{t+\tau-L+1:t+\tau}, \mathbf{Y}_{t+\tau+1:t+\tau+H}). \tag{4}$$

Operationally, the proposed models target window-level changes in location, scale, endpoint drift, range, and first-difference volatility. These statistics do not exhaust all possible forms of nonstationarity, but they correspond to the shifts explicitly exposed to the adaptive normalization and gating modules.

3.2 Mamba Backbone

State space models provide an alternative to attention for sequence modeling. The continuous-time form of a linear state space model is

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \qquad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t), \tag{5}$$

where $\mathbf{h}(t)$ is the latent state, $\mathbf{x}(t)$ is the input, and $\mathbf{y}(t)$ is the output. After discretization, the recurrence can be written abstractly as

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t. \tag{6}$$

Mamba introduces selective state spaces in which the discretization and readout parameters depend on the current input token [15]. The input-dependent update is summarized as

$$\mathbf{h}_t = \bar{\mathbf{A}}_t\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}_t\mathbf{h}_t. \tag{7}$$

This selectivity is useful for forecasting because the model can adapt what information is retained across tokens. Figure 1 summarizes the Mamba block used as the conceptual sequence-modeling backbone.

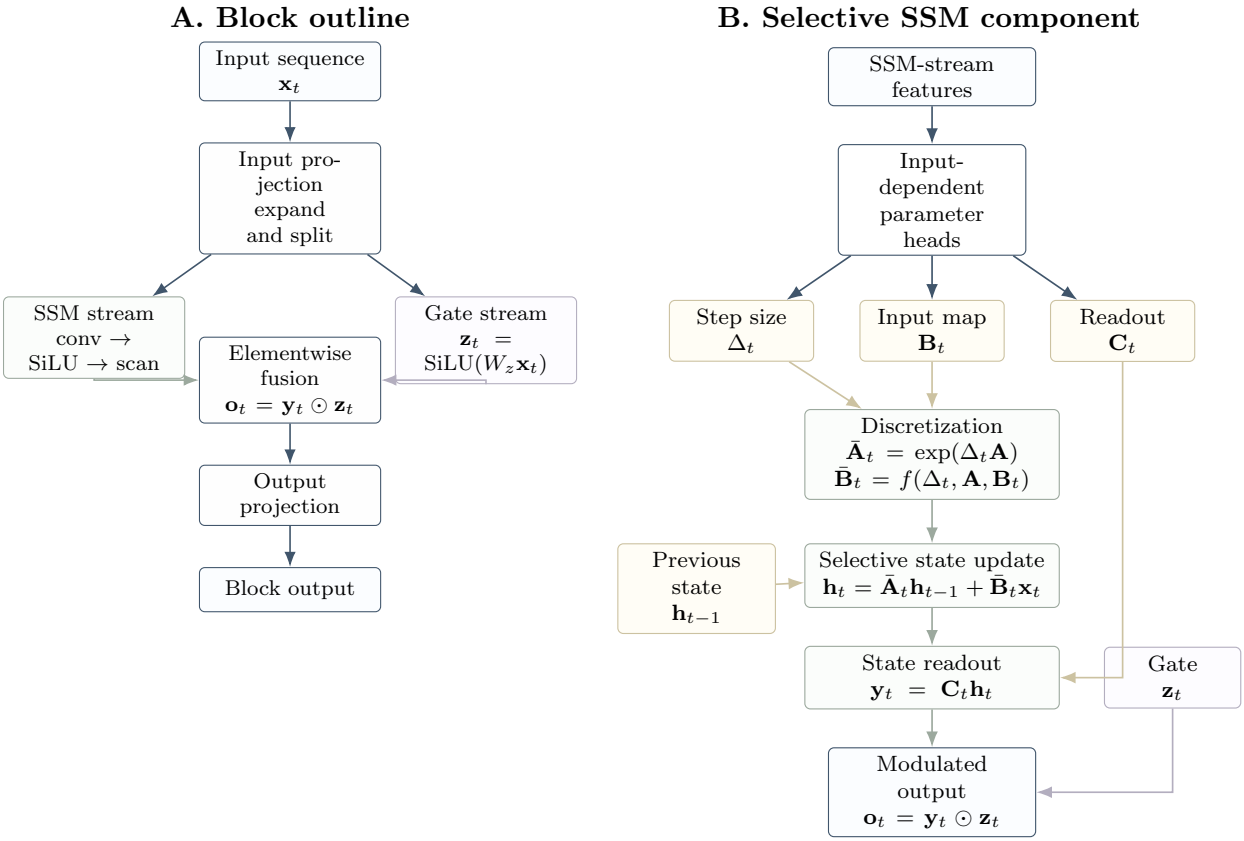


Figure 1: Mamba block used as the sequence-modeling backbone. Panel A shows the high-level block structure with projection, selective state-space processing, gating, fusion, and output projection. Panel B expands the selective state-space component, where input-dependent parameters control the recurrent update and readout.

3.3 S-Mamba Forecasting Encoder

Plain Mamba is a sequence block rather than a complete forecasting model. Given a token sequence $\mathbf{U} \in \mathbb{R}^{B\times T\times d}$, a Mamba block produces another sequence

$$\mathbf{V} = \mathcal{M}(\mathbf{U}) \in \mathbb{R}^{B\times T\times d}, \tag{8}$$

with the same token axis T. For generic sequence modeling, the token axis is usually time. S-Mamba changes this setup for forecasting by using reversible window normalization and inverted variable tokens.

For an input window $\mathbf{X} \in \mathbb{R}^{B\times L\times C}$, S-Mamba first applies reversible normalization:

$$\mu_{b,c} = \frac{1}{L}\sum_{t=1}^{L} X_{b,t,c}, \qquad \sigma_{b,c} = \sqrt{\frac{1}{L}\sum_{t=1}^{L}(X_{b,t,c} - \mu_{b,c})^2 + \epsilon}, \tag{9}$$

$$Z_{b,t,c} = \frac{X_{b,t,c} - \mu_{b,c}}{\sigma_{b,c}}. \tag{10}$$

Each variable history is then embedded as one token:

$$\mathbf{e}_{b,c} = W_{\mathrm{emb}}\mathbf{z}_{b,:,c} + \mathbf{b}_{\mathrm{emb}}, \qquad \mathbf{E} \in \mathbb{R}^{B\times C\times d_{\mathrm{model}}}, \tag{11}$$

where $\mathbf{z}_{b,:,c} \in \mathbb{R}^L$ is the normalized history of variable c. Thus, S-Mamba scans along variable tokens whose features are full lookback histories, rather than scanning directly along time tokens.

Each encoder layer is bidirectional. Let $\mathbf{E}^{(\ell-1)}$ be the input to layer ℓ. The forward and backward Mamba branches are

$$\overrightarrow{\mathbf{H}}^{(\ell)} = \mathcal{M}_f^{(\ell)}(\mathbf{E}^{(\ell-1)}), \qquad \overleftarrow{\mathbf{H}}^{(\ell)} = \mathrm{rev}\left(\mathcal{M}_b^{(\ell)}\left(\mathrm{rev}(\mathbf{E}^{(\ell-1)})\right)\right), \tag{12}$$

and the two directions are added:

$$\mathbf{U}^{(\ell)} = \overrightarrow{\mathbf{H}}^{(\ell)} + \overleftarrow{\mathbf{H}}^{(\ell)}. \tag{13}$$

The implemented layer then applies residual connections, layer normalization, and a feed-forward network:

$$\tilde{\mathbf{E}}^{(\ell)} = \mathrm{LN}_1\left(\mathbf{E}^{(\ell-1)} + \mathbf{U}^{(\ell)}\right), \tag{14}$$

$$\mathbf{E}^{(\ell)} = \mathrm{LN}_2\left(\tilde{\mathbf{E}}^{(\ell)} + \mathrm{FFN}\left(\tilde{\mathbf{E}}^{(\ell)}\right)\right). \tag{15}$$

After the final encoder layer, each variable token is projected to the horizon:

$$\mathbf{P}_{b,c,:} = W_{\mathrm{out}}\mathbf{E}_{b,c,:}^{(L_e)} + \mathbf{b}_{\mathrm{out}}, \qquad \mathbf{P} \in \mathbb{R}^{B\times C\times H}. \tag{16}$$

The forecast is permuted back to the time-major layout and de-normalized:

$$\hat{Y}_{b,h,c} = \sigma_{b,c}P_{b,c,h} + \mu_{b,c}. \tag{17}$$

Figure 2 summarizes the S-Mamba forecasting encoder.

3.4 Nonstationarity Statistics

The NS-Mamba variants expose window statistics to two different modules. Adaptive normalization uses raw window statistics to generate affine parameters, while shift-aware gates use standardized statistics to produce fusion weights. Both are computed from the raw input window before normalization.

For adaptive normalization, the implementation constructs a ten-dimensional statistic vector for each sample and variable:

$$\psi_{b,c} = [\mu_{b,c}, \sigma_{b,c}, X_{b,1,c}, X_{b,L,c}, \mu^r_{b,c}, \sigma^r_{b,c}, X_{b,L,c} - X_{b,1,c}, \min_t X_{b,t,c}, \max_t X_{b,t,c}, \nu_{b,c}], \tag{18}$$

where $\mu_{b,c}$ and $\sigma_{b,c}$ are full-window mean and standard deviation, $\mu^r_{b,c}$ and $\sigma^r_{b,c}$ are computed on the most recent segment of the window, and $\nu_{b,c}$ is the standard deviation of first differences. The recent segment length is controlled by the hyperparameter `ns_recent_ratio`, with default value 0.25.

For shift-aware fusion, the implementation uses an eight-dimensional standardized feature vector:

$$\phi_{b,c} = [\delta_\mu, \delta_\sigma, s, \bar{\nu}, \delta_\nu, r, z_L, z_1]_{b,c}. \tag{19}$$

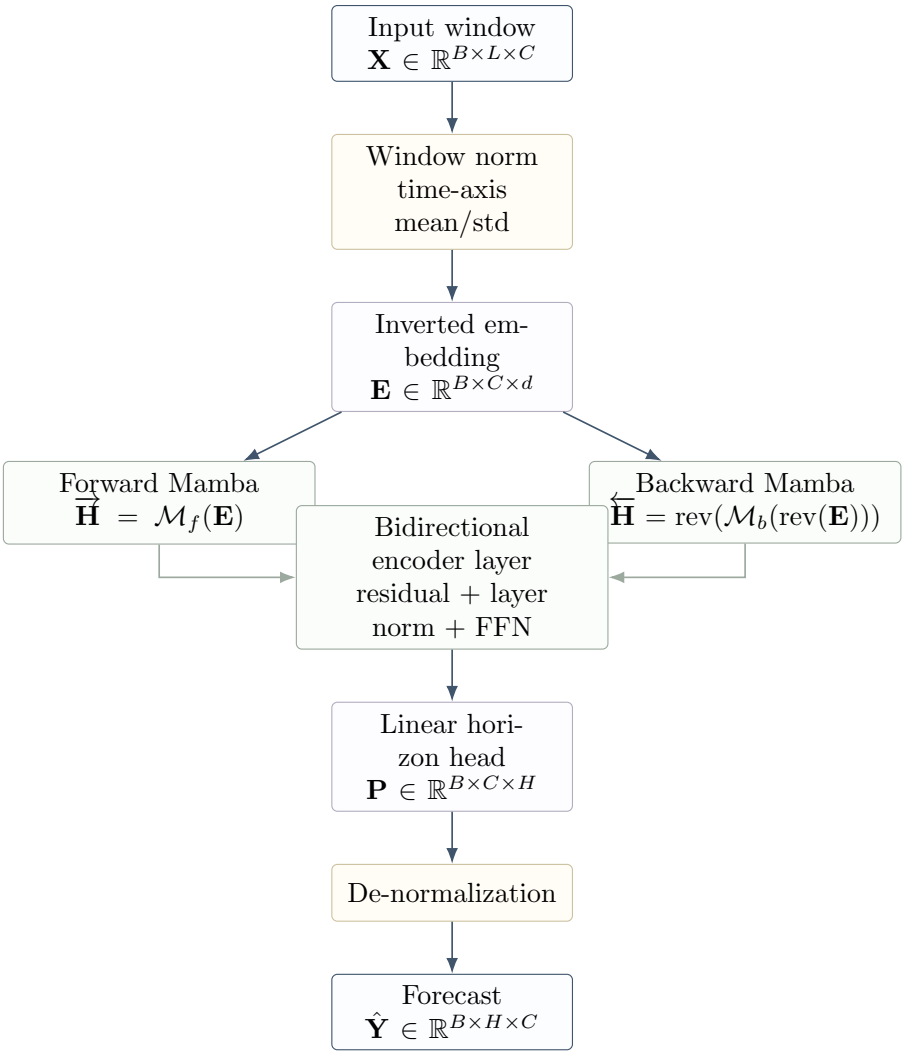


Figure 2: S-Mamba forecasting encoder: reversible normalization, inverted variable-token embedding, bidirectional Mamba encoding, horizon projection, and de-normalization back to the original price scale.

Let $\sigma^+_{b,c} = \sigma_{b,c} + \epsilon$. The entries are

$$\delta_\mu = \frac{\mu^r_{b,c} - \mu_{b,c}}{\sigma^+_{b,c}}, \qquad \delta_\sigma = \log \frac{\sigma^r_{b,c} + \epsilon}{\sigma^+_{b,c}}, \tag{20}$$

$$s = \frac{X_{b,L,c} - X_{b,1,c}}{\sigma^+_{b,c}}, \qquad \bar{\nu} = \frac{\nu_{b,c}}{\sigma^+_{b,c}}, \tag{21}$$

$$\delta_\nu = \log \frac{\nu^r_{b,c} + \epsilon}{\nu_{b,c} + \epsilon}, \qquad r = \frac{\max_t X_{b,t,c} - \min_t X_{b,t,c}}{\sigma^+_{b,c}}, \tag{22}$$

$$z_L = \frac{X_{b,L,c} - \mu_{b,c}}{\sigma^+_{b,c}}, \qquad z_1 = \frac{X_{b,1,c} - \mu_{b,c}}{\sigma^+_{b,c}}. \tag{23}$$

Here $\nu^r_{b,c}$ is the recent-segment standard deviation of first differences. The fusion gates project $\phi_{b,c}$ into the hidden dimension and combine it with the hidden representation when one is available.

When the number of statistic tokens does not match the number of target tokens, the statistic context is averaged over variables and broadcast to the required token count.

3.5 Adaptive Normalization

Adaptive normalization replaces fixed reversible normalization with a learned affine transformation conditioned on $\psi_{b,c}$. For each variable token, an MLP maps $\psi_{b,c}$ to raw affine parameters, and the bounded transformation is

$$\gamma_{b,c} = 1 + a \tanh(g_\gamma(\psi_{b,c})), \qquad \beta_{b,c} = a \tanh(g_\beta(\psi_{b,c})), \tag{24}$$

where a is the adaptive-scale hyperparameter. The normalized input is

$$\tilde{X}_{b,t,c} = \gamma_{b,c} \frac{X_{b,t,c} - \mu_{b,c}}{\sigma_{b,c} + \epsilon} + \beta_{b,c}. \tag{25}$$

The same cached $\gamma_{b,c}$, $\beta_{b,c}$, $\mu_{b,c}$, and $\sigma_{b,c}$ are used to invert the transformation at the output:

$$\hat{Y}_{b,h,c} = \left(\frac{\tilde{Y}_{b,h,c} - \beta_{b,c}}{\gamma_{b,c}} \right) (\sigma_{b,c} + \epsilon) + \mu_{b,c}. \tag{26}$$

3.6 Series Decomposition

The decomposition module separates the normalized window into low-frequency, medium-frequency, and residual components using moving averages. Let $\mathrm{MA}_m(\cdot)$ denote a centered moving average with boundary replication and window size m. The implementation computes

$$\mathbf{Z}^{\mathrm{tr}} = \mathrm{MA}_m(\mathbf{Z}), \tag{27}$$

$$\mathbf{R}^0 = \mathbf{Z} - \mathbf{Z}^{\mathrm{tr}}, \tag{28}$$

$$\mathbf{Z}^{\mathrm{mf}} = \mathrm{MA}_m(\mathbf{R}^0), \tag{29}$$

$$\mathbf{Z}^{\mathrm{re}} = \mathbf{R}^0 - \mathbf{Z}^{\mathrm{mf}}. \tag{30}$$

The superscripts denote low-frequency trend-like, medium-frequency, and residual components. This decomposition is an architectural bias rather than a claim that GLD prices are generated by a perfect additive seasonal process.

3.7 Shift-Aware Fusion

Shift-aware gates use $\phi_{b,c}$ to control either branch fusion or hidden-state interpolation. For decomposition branches with horizon-level outputs $\mathbf{U}_q \in \mathbb{R}^{B \times C \times H}$, where $q \in \{\mathrm{tr}, \mathrm{mf}, \mathrm{re}\}$, the branch gate computes

$$\boldsymbol{\alpha}_{b,c,:,h} = \mathrm{softmax}_q \left(g_\alpha(\phi_{b,c}) \right), \tag{31}$$

and fuses the branch outputs as

$$\mathbf{U}_{b,c,h} = \sum_q \alpha_{b,c,q,h} \mathbf{U}_{q,b,c,h}. \tag{32}$$

The same gate class is also used for scale fusion in the multi-scale models, where the softmax dimension ranges over temporal scales rather than decomposition components. When no decomposition branch is present, a sigmoid gate can interpolate between the encoded hidden state and the input embedding before horizon projection:

$$\tilde{\mathbf{E}} = \mathbf{G} \odot \mathbf{E}_{\mathrm{enc}} + (1 - \mathbf{G}) \odot \mathbf{E}_{\mathrm{in}}. \tag{33}$$

3.8 Variant Definitions

The Mamba-family variant names encode which modules are active. **A** denotes adaptive normalization, **D** denotes decomposition, and **G** denotes shift-aware gated fusion. **M** denotes multi-scale temporal branches, **S** denotes an NLinear-style residual skip, and **P** denotes patch-temporal Mamba branches. All variants produce forecasts with shape $\mathbb{R}^{B\times H\times C}$ and use reversible output scaling when normalization is enabled.

Let $\mathcal{N}(\cdot)$ denote standard reversible normalization and $\mathcal{F}_\theta(\cdot)$ denote one inverted S-Mamba forecasting branch. The S-Mamba baseline is

$$\hat{\mathbf{Y}}_{\mathrm{S}} = \mathcal{N}^{-1}\left(\mathcal{F}_\theta(\mathcal{N}(\mathbf{X}))\right). \tag{34}$$

NS-Mamba-A replaces $\mathcal{N}$ with adaptive normalization. NS-Mamba-D applies standard reversible normalization, decomposes the normalized signal, and sums three independently encoded component forecasts:

$$\hat{\mathbf{Y}}_{\mathrm{D}} = \mathcal{N}^{-1}\left(\sum_{q\in\{\mathrm{tr},\mathrm{mf},\mathrm{re}\}} \mathcal{F}_{\theta_q}(\mathbf{Z}^q)\right). \tag{35}$$

NS-Mamba-AD combines adaptive normalization with decomposition, and NS-Mamba-ADG adds the shift-aware branch gate.

The MS-Mamba family adds temporal scale branches. Let $\mathcal{P}_s(\cdot)$ be average-pooling downsampling by scale s, and let $\mathcal{S}$ be a set of scales such as $\{1,2,4\}$. For a decomposition-enabled multi-scale model, each scale produces

$$\mathbf{U}_s = \sum_{q\in\{\mathrm{tr},\mathrm{mf},\mathrm{re}\}} \mathcal{F}_{\theta_{s,q}}\left(\mathcal{P}_s(\mathbf{Z}^q)\right). \tag{36}$$

MS-Mamba-MD uses mean scale fusion:

$$\hat{\mathbf{Y}}_{\mathrm{MD}} = \mathcal{N}^{-1}\left(\frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \mathbf{U}_s\right), \tag{37}$$

while MS-Mamba-MDG replaces the mean with a shift-aware softmax over scales. MS-Mamba-M and MS-Mamba-MG are the corresponding multi-scale variants without decomposition.

MS-Mamba-MDS adds a last-value residual linear skip:

$$\mathbf{U}_{\mathrm{skip}} = \mathbf{x}_L + \mathbf{W}_{\mathrm{skip}}(\mathbf{Z}-\mathbf{x}_L), \qquad \hat{\mathbf{Y}}_{\mathrm{MDS}} = \mathcal{N}^{-1}\left(\lambda\mathbf{U}_{\mathrm{MD}} + (1-\lambda)\mathbf{U}_{\mathrm{skip}}\right), \tag{38}$$

where $\lambda = \sigma(\rho)$ is learned. PMS-Mamba replaces inverted variable-token branches with patch-temporal Mamba branches. For patch length p and stride r, a scale branch forms patch tokens

$$\mathbf{e}_j^{(s)} = \mathbf{W}_p\,\mathrm{vec}\left(\mathcal{P}_s(\mathbf{Z})_{j:j+p-1}\right), \tag{39}$$

then applies Mamba over the patch index j. PMS-Mamba-PD is the patch-temporal counterpart of MS-Mamba-MD.

3.9 Forecast Head and Loss

The forecast head projects the final hidden representation to H future values. The training objective is mean squared error over the forecast horizon:

$$\mathcal{L} = \frac{1}{BHC_o}\sum_{b=1}^{B}\sum_{h=1}^{H}\sum_{c=1}^{C_o}\left(\hat{Y}_{b,h,c} - Y_{b,h,c}\right)^2. \tag{40}$$

Table 1: Meaning of the Mamba-family variant names used in this manuscript.

Variant	Active modules	Fusion	Purpose
NS-Mamba-A	Adaptive norm	Single path	Test learned normalization
NS-Mamba-D	Decomposition	Sum	Test component separation
NS-Mamba-AD	Adaptive norm + decomposition	Sum	Combine A and D
NS-Mamba-ADG	Adaptive norm + decomposition	Shift gate	Test gated component fusion
MS-Mamba-M	Multi-scale	Mean	Test temporal scales alone
MS-Mamba-MG	Multi-scale	Shift gate	Test gated scale fusion
MS-Mamba-MD	Multi-scale + decomposition	Mean	Promoted Mamba model
MS-Mamba-MDG	Multi-scale + decomposition	Shift gate	Test gated multi-scale decomposition
MS-Mamba-MDS	Multi-scale + decomposition + skip	Mean + skip	Test NLinear-style residual path
PMS-Mamba-PD	Patch Mamba + decomposition	Mean	Test patch Mamba

Inverse transformation is enabled by default in the reported GLD runs, so saved predictions, targets, and metrics are reported on the original GLD price scale unless a run uses `--no-inverse`.

4 Experimental Setup

4.1 Dataset and Preprocessing

The experiments use the bundled GLD daily dataset stored at `data/GLD/GLD.csv`. The file contains 5,313 daily trading observations from 2004-11-18 to 2025-12-31, with columns `date`, `close`, `High`, `Low`, `Open`, and `Volume`. This study uses single-target forecasting with the `close` column as the prediction target, so the current task is univariate-to-univariate forecasting with $C = C_o = 1$. The `close` series has no missing values in the bundled file.

The data loader applies dataset-level standardization with a `StandardScaler` fit on the training split only, and the same scaler is then applied to validation and test windows. At evaluation time, predictions are inverse-transformed back to the original GLD price scale before RMSE and MAE are reported. This dataset-level scaling is separate from the model-level reversible normalization described in Section 3. Date-derived time features are generated from the `date` column with daily frequency and passed to models that use temporal covariates.

4.2 Chronological Data Splits

The GLD series is divided chronologically into 70% training, 10% validation, and 20% testing, following the custom-data split used by the forecasting codebase. For the bundled GLD file, this yields:

- training: 3,719 observations from 2004-11-18 to 2019-08-28;
- validation: 532 observations from 2019-08-29 to 2021-10-07;
- testing: 1,062 observations from 2021-10-08 to 2025-12-31.

Validation and test loaders begin `seq_len` steps before their nominal split boundary so that each target window has a full historical lookback. Because the optimized sweeps use multiple lookback lengths, the exact validation and test context start date depends on the selected `seq_len`. Only the input context overlaps the preceding split; the forecast targets remain inside the validation or test segment. This detail is important for reproducibility because it changes the effective index range seen by the loaders without leaking future targets across split boundaries.

4.3 Evaluation Horizons and Metrics

All experiments use the same forecast horizons

$$H \in \{1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360\}. \tag{41}$$

When grouped summaries are reported, the manuscript uses a balanced 4/4/4 split: short = 1–7 days, middle = 14–90 days, and long = 120–360 days. Let $\mathcal{T}$ denote the set of evaluation windows, and let $|\mathcal{T}|$ be its size. For predicted values $\hat{Y}_{b,h,c}$ and targets $Y_{b,h,c}$, the mean absolute error is

$$\mathrm{MAE} = \frac{1}{|\mathcal{T}|HC_o} \sum_{b\in\mathcal{T}} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left| \hat{Y}_{b,h,c} - Y_{b,h,c} \right|, \tag{42}$$

and the mean squared error is

$$\mathrm{MSE} = \frac{1}{|\mathcal{T}|HC_o} \sum_{b\in\mathcal{T}} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left(\hat{Y}_{b,h,c} - Y_{b,h,c} \right)^2. \tag{43}$$

The root mean squared error is

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}. \tag{44}$$

Because raw RMSE grows with forecast horizon on GLD, we also save the root mean squared scaled error (RMSSE) as a scale-normalized summary for cross-horizon comparison. Let X_t denote the in-sample training series and let the naive denominator use the one-step lag baseline. Then

$$\mathrm{RMSSE} = \sqrt{\frac{\frac{1}{|\mathcal{T}|HC_o} \sum_{b\in\mathcal{T}} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left(\hat{Y}_{b,h,c} - Y_{b,h,c} \right)^2}{\frac{1}{T_{\mathrm{train}}-1} \sum_{t=2}^{T_{\mathrm{train}}} \left(X_t - X_{t-1} \right)^2}}. \tag{45}$$

For completeness, the saved percentage metrics are

$$\mathrm{MAPE} = \frac{100}{|\mathcal{T}|HC_o} \sum_{b\in\mathcal{T}} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left| \frac{\hat{Y}_{b,h,c} - Y_{b,h,c}}{Y_{b,h,c} + \epsilon} \right|, \tag{46}$$

$$\mathrm{MSPE} = \frac{100}{|\mathcal{T}|HC_o} \sum_{b\in\mathcal{T}} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left(\frac{\hat{Y}_{b,h,c} - Y_{b,h,c}}{Y_{b,h,c} + \epsilon} \right)^2, \tag{47}$$

where ϵ is a small stability constant. For interpretation, MAE measures the typical absolute forecast error in GLD price units and is less dominated by a small number of large misses. RMSE is also reported in GLD price units, but because the error is squared before averaging, it penalizes large horizon misses more strongly and is therefore more sensitive to unstable long-range forecasts. RMSSE removes the GLD price scale and rescales the squared forecast error by the average one-step naive training error, which makes it more appropriate than raw RMSE for comparing overall

performance across short, middle, and long horizons. MSE has the same ranking information as RMSE but is less directly interpretable because it is expressed in squared price units. MAPE and MSPE summarize relative error rather than absolute price error; they are useful as supporting diagnostics when comparing horizons with different price levels, but they are not the primary manuscript metrics because the main forecasting question here is how large the price-scale errors are in absolute terms. For the GLD series, which is strictly positive, these percentage metrics are numerically stable, but they can still overweight errors when the denominator is locally small. The primary manuscript metrics are RMSE and MAE on the original GLD price scale. RMSSE, MSE, MAPE, and MSPE are also saved for each run. Training uses mean squared error as the optimization loss, and early stopping is based on validation loss.

4.4 Optimized Evaluation Tracks

The manuscript now reports only hyperparameter-searched result tracks. The first is the refreshed Optuna-tuned non-Mamba baseline sweep. It covers Autoformer, DLinear, Informer, NLinear, Non-stationary Transformer, PatchTST, Reformer, TimesNet, Transformer, and iTransformer. Each model family is tuned with 20 Optuna trials using balanced validation RMSSE over the complete 12-horizon set. The selected configuration is then replayed across all validation horizons and evaluated once on the held-out test split. The resulting summary contains 120 complete best-per-model/per-horizon test rows.

The second track is Mamba-family Optuna search. This search covers S-Mamba, NS-Mamba-D, MS-Mamba-MD, and PMS-Mamba-PD. The completed bundle contains 48 price-scale runs and 48 best-per-model/per-horizon rows, with four selected model-level hyperparameter summaries in Appendix A.2. All four completed Mamba-family variants cover all 12 horizons, so the cross-variant comparison uses both the common long-horizon subset {120, 180, 360} and the promoted long-horizon subset {120, 180, 270, 360}. The current Mamba Optuna summary stores RMSSE, RMSE, and MAE for the selected rows, so RMSSE is used for comparison with the Optuna baseline leaderboard and RMSE/MAE are retained for price-scale interpretation. Detailed sweep presets and architecture-search knobs are reported in Appendix A so the main experimental section can stay focused on the shared data protocol and evaluation design.

4.5 Optimization and Reproducibility

All experiments use the Adam optimizer with one training run per configuration (`itr`=1) and fixed random seed 2023. The epoch limit is 10 and the early-stopping patience is 3 across the optimized sweeps reported here. The Optuna baseline and Mamba-family searches vary model-specific architectural hyperparameters, but keep the same optimizer family, validation-based training loop, and inverse-transformed price-scale evaluation.

For reproducibility, the repository stores run-level prediction arrays, target arrays, and metric arrays together with a run metadata file. Training and validation curves are logged to TensorBoard. The machine-readable result summaries used by the manuscript are bundled under `manuscript/data/` and `results/`.

5 Results

5.1 Optimized GLD Model Comparison

The optimized GLD comparison brings the ten Optuna-tuned non-Mamba baseline families and the four completed Mamba-family variants into one test summary. For each non-Mamba baseline family, 20 Optuna trials were ranked by balanced validation RMSSE over the complete 12-horizon set. The selected baseline configuration was then replayed on the validation horizons and evaluated once on the held-out test split, yielding 120 complete final baseline test rows. The Mamba rows use the completed selected configurations for S-Mamba, NS-Mamba-D, MS-Mamba-MD, and PMS-Mamba-PD, evaluated with the same horizon grouping and final test metrics. The machine-readable result tables are bundled in `manuscript/data/gld_baseline_optuna_best.csv` and `manuscript/data/gld_mamba_optuna_best.csv`; selected hyperparameters are listed in Appendix A.

Table 2: Combined optimized GLD model summary grouped by forecast range. Short = 1–7 days, middle = 14–90 days, long = 120–360 days. Lower RMSSE is better.

Model	Short RMSSE	Middle RMSSE	Long RMSSE	Overall RMSSE	Overall RMSE	Overall MAE
PMS-Mamba-PD	5.30	11.67	27.25	14.74	18.48	12.40
NS-Mamba-D	4.99	12.10	28.63	15.24	19.11	12.81
TimesNet	5.27	12.94	28.78	15.66	19.64	13.30
iTransformer	5.45	13.39	28.21	15.68	19.67	12.84
MS-Mamba-MD	5.06	12.70	30.16	15.98	20.03	13.11
PatchTST	5.13	12.44	30.41	15.99	20.05	13.45
NLinear	4.76	13.22	31.46	16.48	20.66	13.23
S-Mamba	4.78	13.67	31.91	16.79	21.05	13.60
Autoformer	5.45	14.53	32.37	17.45	21.88	14.05
DLinear	5.01	15.05	36.30	18.78	23.55	15.28
Non-stationary Transformer	4.94	16.28	38.68	19.96	25.03	16.62
Transformer	69.21	73.46	74.58	72.42	90.81	57.67
Reformer	67.74	76.00	78.29	74.01	92.80	61.97
Informer	70.95	79.65	75.44	75.35	94.48	61.08

The combined table shows that the strongest overall row is a Mamba-family model rather than a non-Mamba baseline. PMS-Mamba-PD reaches the lowest balanced 12-horizon test RMSSE (14.74), followed by NS-Mamba-D (15.24). Relative to the strongest non-Mamba baseline, PMS-Mamba-PD reduces overall RMSSE from 15.66 to 14.74, a 5.89% reduction. The strongest non-Mamba rows remain competitive: TimesNet is the best baseline overall (15.66), with iTransformer and PatchTST close behind in the upper part of the ranking.

The horizon-range averages show different model strengths rather than a single dominant architecture at every range. NLinear has the best short-range average RMSSE (4.76), while PMS-Mamba-PD leads both the middle range (11.67) and the long range (27.25).

The plain Transformer, Reformer, and Informer configurations remain far weaker on the held-out test split, with overall test RMSSE above 72 despite low validation objectives for some selected trials. This validation-to-test mismatch is why the manuscript reports final test RMSSE and moves the selected hyperparameters to Appendix A for exact reproduction.

5.2 Completed Result Overview

Figure 3 summarizes the currently completed GLD evidence in compact form. Panel A shows the Optuna-tuned non-Mamba baseline frontier using RMSSE, while Panel B shows the completed

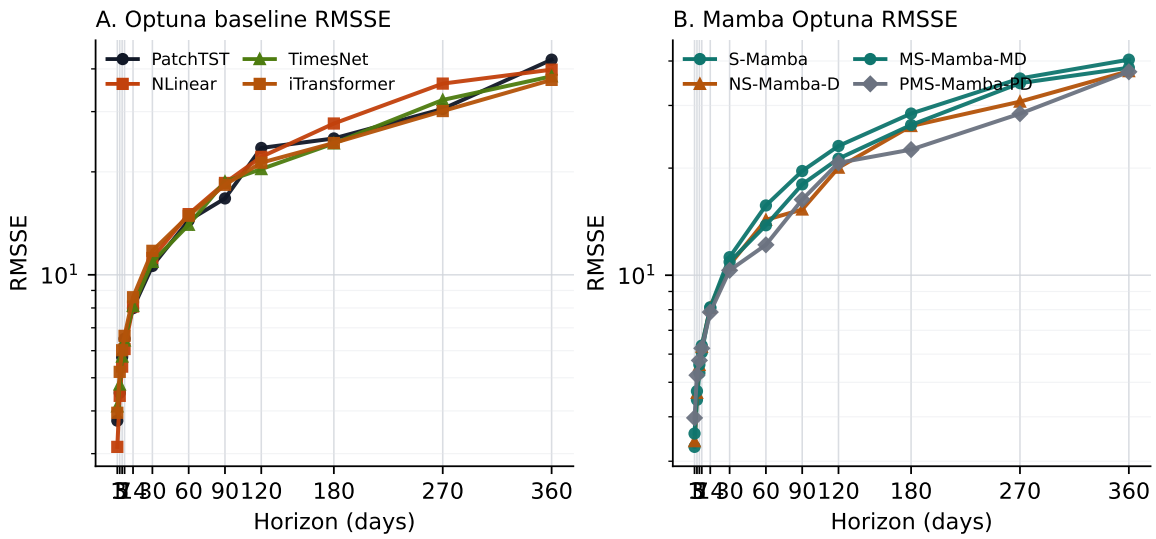


Figure 3: Compact GLD RMSSE summary for the currently completed optimized evaluation tracks. Panel A reports Optuna-tuned non-Mamba baseline RMSSE across all horizons. Panel B reports Mamba Optuna RMSSE across all horizons.

Mamba Optuna variants using the same RMSSE metric across all horizons. The panels use the same scale-normalized metric, so the baseline and Mamba tracks can be read on the same axis.

Taken together, the currently completed optimized evidence supports three main observations. First, Table 2 shows that PMS-Mamba-PD is the strongest overall model across the combined baseline and Mamba comparison, followed by NS-Mamba-D and the best non-Mamba baselines. Second, the horizon-range averages show a trade-off: NLinear remains strongest on the short-range average, while PMS-Mamba-PD leads the middle-range, long-range, and overall summaries. Third, because the Mamba rows now use the same RMSSE definition and horizon grouping as the baseline rows, the comparison is direct: PMS-Mamba-PD reduces average RMSSE by 5.89% against the strongest Optuna-tuned non-Mamba baseline on the full 12-horizon summary.

At the same time, the Mamba rows should be interpreted as the completed GLD Mamba-family search state rather than as a final complete ranking beyond GLD.

6 Discussion and Limitations

This manuscript is currently a draft after the Optuna-tuned non-Mamba baseline sweep and the completed Mamba-family Optuna search. The main limitation is no longer whether strong optimized non-Mamba baselines exist; that part is complete. The remaining limitation is dataset scope. GLD provides a useful nonstationary financial testbed, but one dataset is not enough to establish general forecasting superiority. After GLD experiments are complete, the method should also be tested on standard long-horizon forecasting datasets such as Exchange, Weather, ETT, Electricity, and Traffic.

The second limitation is split sensitivity. Financial time series evaluation can change substantially with different historical periods. The final manuscript should report the exact date range, train-validation-test boundaries, random seeds, and whether validation or test windows use look-back context from earlier splits.

The third limitation is architectural scope. The current Optuna bundle only covers four completed Mamba-family variants, so it supports a clean comparison on GLD but not a broader claim about the entire Mamba design space. The present results suggest that the strongest Mamba configuration depends on lookback selection, moving-average window, and multi-scale structure, so future changes should be justified by improved metrics rather than by architectural novelty alone. The baseline analysis also raises the bar for improvement: any new Mamba variant has to preserve the RMSSE advantage while keeping price-scale RMSE and MAE stable at short, middle, and long horizons.

7 Conclusion

This manuscript presents a structured path for nonstationarity-aware Mamba forecasting. The completed Optuna-tuned GLD baseline sweep shows, under RMSSE-based evaluation, that TimesNet is the strongest overall baseline, with iTransformer a near-tied runner-up, NLinear strongest on short horizons, PatchTST strongest on middle horizons, and TimesNet/iTransformer splitting the long-horizon wins. The proposed NS-Mamba and Mamba architecture families combine decomposition, multi-scale temporal modeling, and Mamba-based sequence processing around an S-Mamba backbone. After Mamba-specific Optuna search, PMS-Mamba-PD becomes the strongest completed Mamba-family model. On the promoted four-horizon summary over 120, 180, 270, and 360 days, it reaches average RMSSE 27.25, average RMSE 34.17, and average MAE 23.72. This optimized Mamba result is slightly stronger than the strongest Optuna-tuned non-Mamba reference on the same RMSSE summary, while the broader claim still remains limited to GLD. The next step is to complete the broader optimized benchmark evaluation with traceable quantitative evidence beyond GLD.

A Appendix

A.1 Sweep Configurations

The refreshed Optuna baseline sweep tunes each non-Mamba model family with 20 trials and selects configurations by balanced validation RMSSE across all 12 horizons. The replayed best rows are stored in `manuscript/data/gld_baseline_optuna_best.csv`, and the selected hyperparameters are summarized below in Appendix A.2. The tuned model families are:

- decomposition and linear baselines: Autoformer, DLinear, and NLinear;
- attention baselines: Informer, Reformer, Transformer, iTransformer, and Non-stationary Transformer;
- patch and multi-period baselines: PatchTST and TimesNet.

The Mamba-family Optuna bundle is stored in `manuscript/data/gld_mamba_optuna_bundle/` and mirrors the extracted reproduction bundle under `scripts/gld_mamba_optuna_bundle/`. The top-level draft inputs are `manuscript/data/gld_mamba_optuna_best.csv`, with selected hyperparameters summarized below in Appendix A.3. The Mamba-family search varies architecture-specific choices such as moving-average window, multi-scale set, fusion rule, linear skip, and patch configuration in addition to lookback length. For the Mamba-family search, the selected model-level configuration is chosen by balanced validation RMSSE, and each reported test row uses the completed replay for that selected configuration.

A.2 Baseline Optuna Hyperparameters

Table 3 reports the selected non-Mamba configuration for each baseline family. *Val.* and *Test* are balanced RMSSE on the validation replay and held-out test split, respectively, so lower values are better. *Seq.* is the lookback length used as model input. The *Training* column gives batch size (`bs`) and learning rate (`lr`). The *Width* column reports `d_model`, the latent representation width, and `d_ff`, the hidden width of the feed-forward or projection block used inside Transformer-style layers. Linear baselines do not use these width parameters, so the table marks them with "–". The final column lists family-specific settings: `factor` controls sparse attention or model-specific expansion settings, `top-k` and `kernels` are TimesNet period-selection settings, `patch` and `stride` define PatchTST tokenization, `moving avg` is the decomposition window, `individual` enables per-channel linear heads, `norm` enables reversible/input normalization, and `projector` gives the Non-stationary Transformer de-stationary projector shape.

The selected baseline configurations show three useful patterns. First, the strongest test rows use either long lookback windows or explicit temporal tokenization: TimesNet uses `seq_len`=256 and reaches the lowest baseline test RMSSE (15.66), iTransformer uses `seq_len`=192 with normalization and is nearly tied (15.68), and PatchTST uses `seq_len`=256 with 32-step patches and stride 16 (15.99). Second, the linear baselines prefer simpler capacity choices: NLinear and DLinear have no `d_model`/`d_ff` width, use the largest learning rate in the selected set (`lr`=5e-3), and differ mainly by decomposition, with DLinear using a 25-step moving-average window. Third, low validation RMSSE is not sufficient for good held-out performance. The plain Transformer has the lowest validation RMSSE in Table 3 (4.93) but a much worse test RMSSE (72.42), and Reformer and Informer show similar validation-to-test degradation. This is why the manuscript treats the validation metric as the selection objective and the test metric as the final comparison result.

Table 3: Selected non-Mamba Optuna configurations for the reported GLD baseline rows. Configurations are selected by balanced validation RMSSE; test RMSSE is reported only for final evaluation. All selected configurations use the study suffix `fullcal_v1`.

Model	Val.	Test	Seq.	Training	Width	Family-specific settings
TimesNet	6.47	15.66	256	bs=32; lr=3e-4	d_model=32; d_ff=128	factor=3; top-k=5; kernels=6
iTransformer	6.48	15.68	192	bs=16; lr=2e-4	d_model=256; d_ff=512	factor=1; norm
PatchTST	6.50	15.99	256	bs=32; lr=2e-4	d_model=256; d_ff=2048	factor=3; patch=32; stride=16
NLinear	6.58	16.48	192	bs=32; lr=5e-3	–	factor=1; individual
Autoformer	6.95	17.45	16	bs=32; lr=2e-4	d_model=256; d_ff=512	factor=3; moving avg=25
DLinear	6.81	18.78	256	bs=32; lr=5e-3	–	factor=1; moving avg=25; individual
Non-stationary Transformer	6.88	19.96	16	bs=16; lr=2e-4	d_model=64; d_ff=128	factor=3; top-k=3; projector=2x256
Transformer	4.93	72.42	192	bs=16; lr=1e-4	d_model=256; d_ff=1024	factor=3
Reformer	11.29	74.01	192	bs=16; lr=3e-4	d_model=512; d_ff=2048	factor=3
Informer	6.82	75.35	16	bs=32; lr=2e-4	d_model=256; d_ff=1024	factor=3

A.3 Mamba Optuna Hyperparameters

Table 4 uses the same reporting convention for the Mamba-family sweep. Here `d_model` is the channel width passed through the Mamba encoder blocks, `d_ff` is the internal projection/feed-forward width around those blocks, and `d_state` is the selective state-space state dimension. The `moving avg` setting controls decomposition smoothing when decomposition is enabled. The `scales` list gives the multi-scale downsampling factors used by MS-Mamba/PMS-Mamba, and `fusion` specifies how scale outputs are merged. `decomp` indicates that trend/seasonal decomposition is active. For PMS-Mamba-PD, `patch` and `stride` define the patching step before multi-scale Mamba processing. `norm` indicates input normalization; `no adaptive norm`, `no shift gate`, and `no linear skip` report disabled optional components in the selected configuration.

The selected Mamba-family hyperparameters separate validation preference from final test performance. NS-Mamba-D has the lowest validation RMSSE (6.32) with `seq_len`=192, `d_model`=128, `d_ff`=256, decomposition, normalization, and no adaptive normalization or shift gate, but its test RMSSE is 15.24. PMS-Mamba-PD has a slightly higher validation RMSSE (6.45) but the best held-out test RMSSE among the Mamba variants (14.74). Its selected configuration uses the longest lookback window (`seq_len`=320), four scales $[1, 2, 4, 8]$, mean fusion, decomposition, 16-step patches with stride 8, and `d_state`=32. MS-Mamba-MD uses a shorter lookback (`seq_len`=96), three scales $[1, 4, 8]$, smaller `d_model`=64, and `d_state`=16, giving test RMSSE 15.98. The S-Mamba backbone is the simplest selected configuration: `seq_len`=32, `d_model`=64, `d_ff`=128, no multi-scale or patch settings, and test RMSSE 16.79.

Compared with the non-Mamba baselines in Appendix A.2, the best Mamba-family test result comes from combining a long lookback, decomposition, multi-scale processing, and patching. PMS-Mamba-PD improves over the strongest non-Mamba baseline in Table 3 by test RMSSE, 14.74 versus 15.66 for TimesNet, while NS-Mamba-D shows that the lowest validation score does not necessarily identify the strongest held-out model.

Table 4: Selected Mamba-family Optuna configurations for the completed GLD bundle. Configurations are selected by balanced validation RMSSE; test RMSSE is reported only for final evaluation.

Model	Val.	Test	Seq.	Training	Width	Architecture settings
PMS-Mamba-PD	6.45	14.74	320	bs=16; lr=2e-4	d_model=128; d_ff=128; d_state=32	factor=1; moving avg=13; scales=[1,2,4,8]; fusion=mean; decomp; patch=16; stride=8; norm
NS-Mamba-D	6.32	15.24	192	bs=16; lr=2e-4	d_model=128; d_ff=256; d_state=32	factor=1; moving avg=13; decomp; norm; no adaptive norm; no shift gate
MS-Mamba-MD	6.82	15.98	96	bs=16; lr=2e-4	d_model=64; d_ff=256; d_state=16	factor=1; moving avg=13; scales=[1,4,8]; fusion=mean; decomp; no linear skip; norm
S-Mamba	6.80	16.79	32	bs=16; lr=3e-4	d_model=64; d_ff=128; d_state=32	factor=1; moving avg=25; norm

References

[1] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.

[2] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.

[3] Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

[4] Jose F. Torres, Dalil Hadjout, Abderrazak Sebaa, Francisco Martinez-Alvarez, and Alicia Troncoso. Deep learning for time series forecasting: A survey. *Big Data*, 9(1):3–21, 2021.

[5] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020.

[6] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. N-hits: Neural hierarchical interpolation for time series forecasting. In *AAAI Conference on Artificial Intelligence*, 2023.

[7] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.

[8] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.

[9] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. In *International Conference on Learning Representations*, 2023.

[10] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90:106181, 2020.

[11] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, 2022.

[12] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.

[13] Tae-Hyun Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.

[14] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Advances in Neural Information Processing Systems*, 2022.

[15] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[16] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is mamba effective for time series forecasting? *arXiv preprint arXiv:2403.11144*, 2024.

[17] Aobo Liang, Xingguo Jiang, Yan Sun, Xiaohou Shi, and Ke Li. Bi-mamba+: Bidirectional mamba for time series forecasting. *arXiv preprint arXiv:2404.15772*, 2024.

[18] Xiuding Cai, Yaoyao Zhu, Xueyao Wang, and Yu Yao. Mambats: Improved selective state space models for long-term time series forecasting. *arXiv preprint arXiv:2405.16440*, 2024.

[19] Md Atik Ahamed and Qiang Cheng. Timemachine: A time series is worth 4 mambas for long-term forecasting. In *European Conference on Artificial Intelligence*, 2024.

[20] Haoyu Ma, Yushu Chen, Wenlai Zhao, Jinzhe Yang, Yingsheng Ji, Xinghua Xu, Xiaozhu Liu, Hao Jing, Shengzhuo Liu, and Guangwen Yang. A mamba foundation model for time series forecasting. *arXiv preprint arXiv:2411.02941*, 2024.

[21] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The m4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020.

[22] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.